エンジニア研修 ・ 2026年度版

Stream API

「どうやるか」ではなく「何をしたいか」を書く

中間操作 終端操作 Collectors 遅延評価

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

この講義の流れ

Stream API を学び直す

	テーマ	内容
01	Stream API を使う理由	可読性と不変性
02	中間操作と終端操作	終端操作まで実行されない
03	メソッドの全体像	中間操作・終端操作の一覧
04	中間操作	filter・map・sorted ほか
05	終端操作	anyMatch・reduce・collect ほか
06	Stream の組み合わせ演習	
	まとめ	やりたいこと別 早見表

各メソッドの説明のあとに小さな演習があります。サンプルは JShell で実行し、出力を自分の目で確かめながら進めます。

はじめに

演習準備：Docker経由でJShellを使う

サンプルの実行と演習では、Docker経由でJShellを使用します。以下のコマンドでJShellを実行できるようにしておきます。

```
docker container run --name edu2026_stream -ti --rm amazoncorretto:25 jshell
```

JShellとは

Java Shellの略で、JavaのREPL環境としてJava9以降に導入された対話型ツールです。

REPLとは

Read-Eval-Print Loopの略で、入力内容をその場で読み込み（Read）評価し（Eval）表示する（Print）を繰り返す実行環境のことで、小さなプログラムの実行確認などに使用されるものです。

はじめに

複数行のコードは { } で囲んで貼り付ける

JShell は、1行ごとに「ここで文が終わっているか」を判断して実行します。行頭が「.」で始まるつなぎ方は、1行目の時点で文が終わったと判断されてしまいます。このスライド以降のサンプルは、すべて { } で囲んだ形で載せているので、そのままコピーして貼り付けてください。

そのまま貼り付けると

```
List<String> names = trainees.stream()
    .map(Trainee::name)
    .toList();
```

JShell の出力

```
|  Error:
|  incompatible types:
java.util.stream.Stream<Trainee> cannot be converted
to java.util.List<java.lang.String>
（以下略。2行目・3行目もエラーになる）
```

{ } で囲んで貼り付けると

```
{
    List<String> names = trainees.stream()
        .map(Trainee::name)
        .toList();
    System.out.println(names);
}
```

JShell の出力

```
[佐藤, 鈴木, 高橋, 田中, 伊藤, 渡辺]
```

はじめに

サンプルデータを JShell に読み込む

最初に、次のコードを JShell に貼り付けておきます。JShell を終了したら、もう一度貼り付けます。

```
record Trainee(String name, String team, int score) {}

var trainees = List.of(
    new Trainee("佐藤", "A", 82),
    new Trainee("鈴木", "B", 65),
    new Trainee("高橋", "A", 91),
    new Trainee("田中", "C", 48),
    new Trainee("伊藤", "B", 77),
    new Trainee("渡辺", "C", 70));
```

読み方

- record は、値を持つだけのクラスを短く書ける構文
- t.name()、t.team()、t.score() で各値を取り出せる
- 以降のサンプルは、すべてこの trainees を使う

01

Stream API を使う理由

やりたいことを、メソッド名で書く

Stream API を使う理由

70点以上の研修生の名前を集める、2つの書き方

for-if で書く

```
{
    List<String> names = new ArrayList<>();
    for (Trainee t : trainees) {
        if (t.score() >= 70) {
            names.add(t.name());
        }
    }
    System.out.println(names);
}
```

実行結果

[佐藤, 高橋, 伊藤, 渡辺]

Stream API で書く

```
{
    List<String> names = trainees.stream()
        .filter(t -> t.score() >= 70)
        .map(t -> t.name())
        .toList();
    System.out.println(names);
}
```

実行結果

[佐藤, 高橋, 伊藤, 渡辺]

どちらも結果は同じ。違うのは、読み手が「何をしているか」をつかむまでの手間です。

Stream API を使う理由－可読性

for-if は「どうやるか」、Stream は「何をしたいか」を書く

for-if を読むときは

「空のリストを作って」「1件ずつ見て」「条件が合えば追加して」…と、手順を頭の中でなぞってから、ようやく「70点以上の名前を集めている」と分かる。

Stream を読むときは

filter（絞り込む）→ map（変換する）→ toList（まとめる）。メソッド名がそのまま「やりたいこと」を言語化しているので、上から読むだけで意図が分かる。

for-if で書いていた手順	Stream で書くと	言いたいこと
if (条件) { list.add(x); }	filter	条件で絞り込む
list.add(x.getName());	map	別の値に変換する
if (条件) { found = true; break; }	anyMatch	1つでも当てはまるか
sum += x.getScore();	mapToInt / sum	合計する
map.computeIfAbsent(k, …).add(x);	groupingBy	キーごとに分類する

Stream API を使う理由－不変性

元のデータも、途中の変数も書き換えない

for-if：変数を書き換えながら結果を作る

```
{
    List<String> names = new ArrayList<>();
    int total = 0;
    for (Trainee t : trainees) {
        if (t.score() >= 70) {
            names.add(t.name()); // リストを書き換える
            total += t.score(); // 変数を書き換える
        }
    }
    System.out.println(names + " " + total);
}
```

実行結果

```
[佐藤, 高橋, 伊藤, 渡辺] 320
```

Stream：結果を新しい値として受け取る

```
{
    List<String> names = trainees.stream()
        .filter(t -> t.score() >= 70)
        .map(t -> t.name())
        .toList();
    int total = trainees.stream()
        .filter(t -> t.score() >= 70)
        .mapToInt(t -> t.score())
        .sum();
    System.out.println(names + " " + total);
}
```

実行結果

```
[佐藤, 高橋, 伊藤, 渡辺] 320
```

Stream API を使う理由－不変性

書き換えないことで得られること

書き換える場所がない

names や total に途中の状態が無いので、「この時点で中身はどうなっているか」を追いかける必要がない。

元のリストは変わらない

trainees はそのまま。同じ trainees から、別の集計を何度でも安全に始められる。

結果も変更できない

toList() が返すリストは変更不可。受け取った後で誰かに書き換えられる心配がない。

Stream API を使う理由－不変性

ラムダ式の中で、外の変数を書き換えない

ラムダ式の中からは、外で宣言したローカル変数を書き換えられません。次のメソッドは JShell に貼り付けた時点でエラーになります。

```
void countPassed() {
    int count = 0;
    trainees.stream()
        .filter(t -> t.score() >= 70)
        .forEach(t -> count++); // コンパイルエラー
}
```

JShell の出力

```
|  Error:
|  local variables referenced from a lambda expression must be final or effectively final
|  …（エラー箇所の表示は省略）
```

Stream API を使う理由－不変性

数えたいなら、count で結果として受け取る

```
{
    long count = trainees.stream()
        .filter(t -> t.score() >= 70)
        .count();
    System.out.println(count);
}
```

実行結果

4

外の変数を書き換えたくなったら、「その値を戻り値で受け取れないか」を考える。

読み方

ラムダ式とメソッド参照

ラムダ式

```
t -> t.score() >= 70
```

「t を受け取り、t.score() >= 70 の結果を返す処理」。名前のないメソッドを、その場で渡しているもの。引数が2つなら (sum, score) -> sum + score のように ( ) で囲む。

メソッド参照

```
Trainee::name
```

t -> t.name() と同じ意味。受け取った値のメソッドを呼ぶだけのラムダ式は、こう短く書ける。System.out::println は x -> System.out.println(x) と同じ。

読み方

メソッドが受け取る「処理」の型

Stream のメソッドの引数には、ラムダ式で書いた処理を渡します。引数の型を見ると、どんな処理を渡せばよいかが分かります。

引数の型	渡す処理	使うメソッドの例
Predicate<T>	T を受け取り、boolean を返す	filter、anyMatch、allMatch
Function<T, R>	T を受け取り、R を返す（変換）	map、flatMap
Comparator<T>	T を2つ受け取り、大小を返す	sorted、max、min
BinaryOperator<T>	T を2つ受け取り、T を返す	reduce
Consumer<T>	T を受け取り、何も返さない	forEach、ifPresent
Supplier<T>	何も受け取らず、T を返す	orElseGet、or

※ スライドでは、ジェネリクスの ? super / ? extends を省略して表記します。

02

中間操作と終端操作

終端操作を呼ぶまで、何も実行されない

中間操作と終端操作

Stream は「生成 → 中間操作 → 終端操作」の流れでできている

生成

`trainees.stream()`

リストなどから Stream を作る

→

中間操作

`.filter(…)`

Stream を返す

→

中間操作

`.map(…)`

Stream を返す

→

終端操作

`.toList()`

Stream 以外を返す

中間操作

戻り値の型が Stream。だから、続けて次のメソッドをつなげられる。何個つないでもよい。

終端操作

戻り値の型が Stream 以外（List、boolean、Optional、void など）。ここで流れが終わり、処理が実行される。

迷ったら戻り値の型を見る。Stream を返すなら中間操作、それ以外なら終端操作です。

中間操作と終端操作ー遅延評価

終端操作を呼ぶまで、中間操作は実行されない（1/2）

filter と map の中に println を入れて、いつ実行されるかを観察します。

```
{
    Stream<String> s = trainees.stream()
        .filter(t -> {
            System.out.println("filter: " + t.name());
            return t.score() >= 70;
        })
        .map(t -> {
            System.out.println("map: " + t.name());
            return t.name();
        });
    System.out.println("--- 終端操作の前 ---");
    List<String> names = s.toList();
    System.out.println(names);
}
```

終端操作を呼ぶまで、中間操作は実行されない（2/2）

実行結果

```
--- 終端操作の前 ---
filter: 佐藤
map: 佐藤
filter: 鈴木
filter: 高橋
map: 高橋
filter: 田中
filter: 伊藤
map: 伊藤
filter: 渡辺
map: 渡辺
[佐藤, 高橋, 伊藤, 渡辺]
```

ポイント

- 「--- 終端操作の前 ---」が最初に出力される
- toList() を呼んだ時点で、filter と map が動き出す

中間操作と終端操作－遅延評価

操作ごとではなく、要素ごとに処理する

要素（順番に流れる）	filter（70点以上？）	map（名前に変換）	toList
佐藤	82 ✓	佐藤	追加
鈴木	65 ✗	—	—
高橋	91 ✓	高橋	追加
田中	48 ✗	—	—
伊藤	77 ✓	伊藤	追加
渡辺	70 ✓	渡辺	追加

前のスライドの出力の読み方

- 「全件を filter → 全件を map」ではなく、1件ずつ filter → map → toList と流れる
- 条件に合わなかった要素は、その場で捨てられ、map は実行されない
- 途中結果のリストを作らないので、無駄な処理やメモリを使わない

中間操作と終端操作ー遅延評価

答えが決まった時点で、残りの要素は処理しない

```
{
    Optional<String> first = trainees.stream()
        .filter(t -> {
            System.out.println("check: " + t.name());
            return t.score() >= 90;
        })
        .map(t -> t.name())
        .findFirst();
    System.out.println(first);
}
```

実行結果

```
check: 佐藤
check: 鈴木
check: 高橋
Optional[高橋]
```

答えが決まった時点で、残りの要素は処理しない（ポイント）

ポイント

・6人のうち、高橋さんまでの3人しか調べていない（findFirst は1件見つかれば終わる）

・anyMatch・allMatch・limit なども同じように途中で止まる

中間操作と終端操作

Stream は一度しか使えない

```
{
    Stream<Trainee> s = trainees.stream();
    System.out.println(s.count());
    System.out.println(s.count()); // 2回目
}
```

実行結果

```
6
|  Exception java.lang.IllegalStateException: stream has already been operated upon or closed
|        …（スタックトレースは省略）
```

ポイント

- 終端操作を呼んだ Stream は使い終わり。もう一度処理したいときは、trainees.stream() から作り直す
- Stream を変数にしまっておかず、生成から終端操作までを一続きに書くのが基本

演習 ・ 中間操作と終端操作

出力される内容を予想する

次のコードを実行したとき、何がどの順番で出力されるかを紙に書いて予想し、実行して答え合わせをしてください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

```
{
    Stream<String> s = trainees.stream()
        .filter(t -> {
            System.out.println("filter: " + t.name());
            return t.team().equals("B");
        })
        .map(t -> {
            System.out.println("map: " + t.name());
            return t.name();
        });
    System.out.println("start");
    System.out.println(s.findFirst());
}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作と終端操作

出力される内容を予想する

```
{
    Stream<String> s = trainees.stream()
        .filter(t -> {
            System.out.println("filter: " + t.name());
            return t.team().equals("B");
        })
        .map(t -> {
            System.out.println("map: " + t.name());
            return t.name();
        });
    System.out.println("start");
    System.out.println(s.findFirst());
}
```

実行結果

```
start
filter: 佐藤
filter: 鈴木
map: 鈴木
Optional[鈴木]
```

解答例 ・ 中間操作と終端操作

出力される内容を予想する（解説）

解説

・終端操作 findFirst を呼ぶまで、filter も map も動かない

・鈴木さん（Bチーム）が見つかった時点で終わるので、高橋さん以降は調べない

03

メソッドの全体像

よく使うメソッドを、中間操作と終端操作に分けて見る

メソッドの全体像ー中間操作

中間操作：Stream を受け取り、Stream を返す

メソッド	役割
filter	条件に合う要素だけ残す
map	各要素を別の値に変換する
sorted	並べ替える
distinct	重複を取り除く
limit / skip	先頭から切り出す・飛ばす
flatMap	ネストしたリストを平らにする
mapToInt	数値専用の IntStream にする

どれも戻り値は Stream。だから「.」で次々とつなげられます。

メソッドの全体像ー終端操作

終端操作：Stream を受け取り、Stream 以外を返す

メソッド	役割	戻り値
anyMatch / allMatch / noneMatch	条件に当てはまるか判定する	boolean
count	件数を数える	long
findFirst / min / max	1件だけ取り出す	Optional<T>
sum / average	合計・平均（IntStream）	int / OptionalDouble
reduce	1つの値に畳み込む	T
toList	List にまとめる	List<T>
collect	Collectors で好きな形にまとめる	String、Map など
forEach	1件ずつ処理する	void

04

中間操作

Stream を返すので、つなげて書ける

中間操作 ・ filter

filter：条件に合う要素だけを残す

条件を満たす（true を返す）要素だけを残し、それ以外を捨てます。

```
Stream<T> filter(Predicate<T> predicate)
```

```
{
    List<Trainee> over80 = trainees.stream()
        .filter(t -> t.score() >= 80)
        .toList();
    System.out.println(over80);
}
```

実行結果

```
[Trainee[name=佐藤, team=A, score=82], Trainee[name=高橋, team=A, score=91]]
```

ポイント

- for-if の「if」にあたる
- 元の trainees は変わらない
- 条件は boolean を返すラムダ式で書く

演習 ・ 中間操作 ・ filter

Cチームの研修生だけを取り出す

trainees から、Cチーム（team が "C"）の研修生だけを取り出してリストにし、そのリストを出力してください。

ヒント：文字列の比較は == ではなく equals を使う。リストにするには toList()

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[Trainee[name=田中, team=C, score=48], Trainee[name=渡辺, team=C, score=70]]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を {} で囲んで貼り付けます。

解答例 ・ 中間操作 ・ filter

Cチームの研修生だけを取り出す

```
{
    List<Trainee> teamC = trainees.stream()
        .filter(t -> t.team().equals("C"))
        .toList();
    System.out.println(teamC);
}
```

実行結果

```
[Trainee[name=田中, team=C, score=48], Trainee[name=渡辺, team=C, score=70]]
```

解説

・t.team().equals("C") で文字列を比較する

・toList() でリストにまとめてから、System.out.println で出力する

中間操作 ・ map

map：各要素を別の値に変換する（1/2）

各要素に変換処理を適用し、その結果を新しい要素にします。要素の数は変わりません。

```
Stream<R> map(Function<T, R> mapper)
```

```
{
    List<String> names = trainees.stream()
        .map(Trainee::name)
        .toList();
    System.out.println(names);
}
```

実行結果

[佐藤, 鈴木, 高橋, 田中, 伊藤, 渡辺]

map：各要素を別の値に変換する（2/2）

```
{
    List<Integer> scores = trainees.stream()
        .map(t -> t.score())
        .toList();
    System.out.println(scores);
}
```

実行結果

```
[82, 65, 91, 48, 77, 70]
```

ポイント

- Trainee から String へ、のように型が変わってもよい
- Trainee::name は t -> t.name() と同じ
- for-if で「別の値を詰め直す」部分にあたる

演習 ・ 中間操作 ・ map

「チーム-名前」の文字列に変換する

trainees の各研修生を、「A-佐藤」のような「チーム-名前」形式の文字列に変換し、リストにして出力してください。

ヒント：文字列は + でつなげられる

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[A-佐藤, B-鈴木, A-高橋, C-田中, B-伊藤, C-渡辺]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ map

「チーム-名前」の文字列に変換する

```
{
    List<String> labels = trainees.stream()
        .map(t -> t.team() + "-" + t.name())
        .toList();
    System.out.println(labels);
}
```

実行結果

```
[A-佐藤, B-鈴木, A-高橋, C-田中, B-伊藤, C-渡辺]
```

解説

- Trainee を受け取り String を返すラムダ式を渡す
- 要素数は6件のまま変わらない

中間操作 ・ sorted

sorted：要素を並べ替える

並べ方（Comparator）を指定して並べ替えます。引数なしの sorted() は、数値や文字列の自然な順序で並べます。

```
Stream<T> sorted(Comparator<T> comparator)
```

```
{
    List<String> ranking = trainees.stream()
        .sorted(Comparator.comparing(Trainee::score).reversed())
        .map(t -> t.name() + ":" + t.score())
        .toList();
    System.out.println(ranking);
}
```

実行結果

```
[高橋:91, 佐藤:82, 伊藤:77, 渡辺:70, 鈴木:65, 田中:48]
```

中間操作 ・ sorted

sorted：要素を並べ替える（ポイント）

ポイント

・Comparator.comparing(取り出す値) で、何を基準に並べるかを書く

・.reversed() で降順

・.thenComparing(…) で第2の基準を足せる

演習 ・ 中間操作 ・ sorted

点数の低い順に名前を並べる

trainees を点数の低い順（昇順）に並べ替え、名前のリストにして出力してください。

ヒント：昇順なら reversed() は不要

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

[田中，鈴木，渡辺，伊藤，佐藤，高橋]

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ sorted

点数の低い順に名前を並べる

```
{
    List<String> names = trainees.stream()
        .sorted(Comparator.comparing(Trainee::score))
        .map(Trainee::name)
        .toList();
    System.out.println(names);
}
```

実行結果

[田中, 鈴木, 渡辺, 伊藤, 佐藤, 高橋]

解説

・並べ替えてから map で名前に変換する

・map を先にすると、点数が分からなくなり並べられない

中間操作 ・ distinct

distinct：重複する要素を取り除く

同じ値（equals が true）の要素を1つにまとめます。最初に現れた順番は保たれます。

`Stream<T> distinct()`

```
{
    List<String> teams = trainees.stream()
        .map(Trainee::team)
        .distinct()
        .toList();
    System.out.println(teams);
}
```

実行結果

```
[A, B, C]
```

ポイント

・チーム名のように、重複のある値を一覧にしたいときに使う

・record はフィールドの値がすべて同じなら equals が true

演習 ・ 中間操作 ・ distinct

点数帯の一覧を作る

各研修生の点数を10点単位に切り捨て（82点なら80）、重複を取り除いて、小さい順のリストにして出力してください。

ヒント：int どうしの割り算は小数点以下が切り捨てられる（82 / 10 * 10 は 80）。小さい順は sorted()

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[40, 60, 70, 80, 90]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ distinct

点数帯の一覧を作る

```
{
    List<Integer> ranges = trainees.stream()
        .map(t -> t.score() / 10 * 10)
        .distinct()
        .sorted()
        .toList();
    System.out.println(ranges);
}
```

実行結果

```
[40, 60, 70, 80, 90]
```

解説

- map で点数帯に変換してから distinct する
- Integer は自然な順序があるので sorted() だけで並ぶ

中間操作 ・ limit / skip

limit / skip：先頭から切り出す・飛ばす（1/2）

limit(n) は先頭から n 件だけを残し、skip(n) は先頭の n 件を飛ばします。

`Stream<T> limit(long n) ／ Stream<T> skip(long n)`

```
{
    List<String> top3 = trainees.stream()
        .sorted(Comparator.comparing(Trainee::score).reversed())
        .limit(3)
        .map(Trainee::name)
        .toList();
    System.out.println(top3);
}
```

実行結果

[高橋, 佐藤, 伊藤]

中間操作 ・ limit / skip

limit / skip：先頭から切り出す・飛ばす（2/2）

```
{
    List<String> rest = trainees.stream()
        .skip(4)
        .map(Trainee::name)
        .toList();
    System.out.println(rest);
}
```

実行結果

[伊藤, 渡辺]

ポイント

- sorted と組み合わせると「上位3人」が書ける
- limit は必要な件数がそろうと処理を止める
- ページ分けは skip(ページ番号 × 件数).limit(件数)

演習 ・ 中間操作 ・ limit / skip

2ページ目に表示する研修生

trainees を2件ずつのページに分けたとき、2ページ目（3件目と4件目）に表示される研修生の名前をリストにして出力してください。

ヒント：1ページ目の2件を飛ばしてから、2件だけ残す

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

[高橋，田中]

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ limit / skip

2ページ目に表示する研修生

```
{
    List<String> page2 = trainees.stream()
        .skip(2)
        .limit(2)
        .map(Trainee::name)
        .toList();
    System.out.println(page2);
}
```

実行結果

[高橋，田中]

解説

・skip(2) で1ページ目を飛ばし、limit(2) で2件に絞る

・skip と limit の順番を逆にすると結果が変わる

中間操作 ・ flatMap

flatMap：ネストしたリストを平らにする

各要素を Stream に変換し、それらを1本の Stream につなぎます。List の中に List があるような入れ子のデータを、平らにならすときに使います。

```
Stream<R> flatMap(Function<T, Stream<R>> mapper)
```

```
{
    List<List<String>> skillsByTrainee = List.of(
        List.of("Java", "SQL"),
        List.of("Java"),
        List.of("HTML", "CSS", "Java"));

    List<String> skills = skillsByTrainee.stream()
        .flatMap(list -> list.stream())
        .distinct()
        .toList();
    System.out.println(skills);
}
```

実行結果

```
[Java, SQL, HTML, CSS]
```

中間操作 ・ flatMap

flatMap：ネストしたリストを平らにする（ポイント）

ポイント

・map だと Stream<List<String>> のように入れ物が残る

・list -> list.stream() で中身を取り出して平らにする

演習 ・ 中間操作 ・ flatMap

文から単語の一覧を作る

List.of("Java SQL", "Java HTML CSS", "SQL Git") の各文字列を空白で単語に分け、重複を取り除いた単語のリストを出力してください。

ヒント：line.split(" ") で String[] になる。配列は Arrays.stream(配列) で Stream にできる

期待する結果

```
[Java, SQL, HTML, CSS, Git]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ flatMap

文から単語の一覧を作る

```
{
    List<String> lines = List.of("Java SQL", "Java HTML CSS", "SQL Git");

    List<String> words = lines.stream()
        .flatMap(line -> Arrays.stream(line.split(" ")))
        .distinct()
        .toList();
    System.out.println(words);
}
```

実行結果

```
[Java, SQL, HTML, CSS, Git]
```

解説

・split すると各文字列が単語の配列になり、配列が入れ子になる。flatMap で平らにして、単語1本の Stream にする

・distinct で、2回出てくる Java と SQL を1つにする

中間操作 ・ mapToInt

mapToInt：数値専用の IntStream にする（1/2）

各要素を int に変換し、IntStream にします。IntStream には sum・average・max などの集計メソッドがあります。

```
IntStream mapToInt(ToIntFunction<T> mapper)
```

```
{
    int total = trainees.stream()
        .mapToInt(Trainee::score)
        .sum();
    System.out.println(total);
}
```

実行結果

```
433
```

中間操作 ・ mapToInt

mapToInt：数値専用の IntStream にする（2/2）

```
{
    double avg = trainees.stream()
        .mapToInt(Trainee::score)
        .average()
        .orElse(0.0);
    System.out.println(avg);
}
```

実行結果

```
72.16666666666667
```

ポイント

・Stream<Integer> には sum が無いので、集計するときは mapToInt に切り替える

・average は要素0件のこともあるので OptionalDouble を返す。orElse で値を取り出す

演習 ・ 中間操作 ・ mapToInt

Aチームの合計点

Aチームの研修生の点数の合計を求めて出力してください。

ヒント：filter で絞り込んでから mapToInt、最後に sum

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

173

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 中間操作 ・ mapToInt

Aチームの合計点

```
{
    int totalA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .mapToInt(Trainee::score)
        .sum();
    System.out.println(totalA);
}
```

実行結果

173

解説

・filter → mapToInt → sum の順に、絞り込み・変換・集計をつなぐ

05

終端操作

Stream 以外を返し、ここで処理が実行される

終端操作 ・ anyMatch

anyMatch：1つでも条件に当てはまるか（1/2）

条件を満たす要素が1つでもあれば true を返します。

```
boolean anyMatch(Predicate<T> predicate)
```

```
{
    boolean hasFailure = trainees.stream()
        .anyMatch(t -> t.score() < 60);
    System.out.println(hasFailure);
}
```

実行結果

```
true
```

終端操作 ・ anyMatch

anyMatch：1つでも条件に当てはまるか（2/2）

```
{
    boolean hasTeamD = trainees.stream()
        .anyMatch(t -> t.team().equals("D"));
    System.out.println(hasTeamD);
}
```

実行結果

```
false
```

ポイント

- 見つかった時点で残りは調べない
- for + if + break + フラグ変数の組み合わせを1行で書ける
- 要素が0件なら false

演習 ・ 終端操作 ・ anyMatch

90点以上の研修生はいるか

90点以上の研修生が1人でもいるかを判定し、結果を出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
true
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ anyMatch

90点以上の研修生はいるか

```
{
    boolean excellent = trainees.stream()
        .anyMatch(t -> t.score() >= 90);
    System.out.println(excellent);
}
```

実行結果

```
true
```

解説

・「いるか？」は anyMatch

・高橋さん（91点）が見つかった時点で終わる

終端操作 ・ allMatch

allMatch：すべてが条件に当てはまるか（1/2）

すべての要素が条件を満たすときだけ true を返します。

```
boolean allMatch(Predicate<T> predicate)
```

```
{
    boolean allPassed = trainees.stream()
        .allMatch(t -> t.score() >= 60);
    System.out.println(allPassed);
}
```

実行結果

```
false
```

終端操作 ・ allMatch

allMatch：すべてが条件に当てはまるか（2/2）

```
{
    boolean allOver40 = trainees.stream()
        .allMatch(t -> t.score() >= 40);
    System.out.println(allOver40);
}
```

実行結果

```
true
```

ポイント

・条件を満たさない要素が見つかった時点で false が決まり、止まる

・要素が0件なら true になる（当てはまらない要素が1つも無いため）。filter の後に使うときは注意

演習 ・ 終端操作 ・ allMatch

Aチーム全員が80点以上か

Aチームの研修生が全員80点以上かどうかを判定し、結果を出力してください。

ヒント：まず filter で Aチームに絞り込む

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
true
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ allMatch

Aチーム全員が80点以上か

```
{
    boolean teamAOver80 = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .allMatch(t -> t.score() >= 80);
    System.out.println(teamAOver80);
}
```

実行結果

```
true
```

解説

・filter で対象を絞ってから allMatch で判定する

・佐藤さん82点、高橋さん91点なので true

終端操作 ・ noneMatch

noneMatch：どれも条件に当てはまらないか（1/2）

どの要素も条件を満たさないときに true を返します。anyMatch の結果を反対にしたものと同じです。

`boolean noneMatch(Predicate<T> predicate)`

```
{
    boolean noPerfect = trainees.stream()
        .noneMatch(t -> t.score() == 100);
    System.out.println(noPerfect);
}
```

実行結果

```
true
```

終端操作 ・ noneMatch

noneMatch：どれも条件に当てはまらないか（2/2）

```
{
    boolean noUnder50 = trainees.stream()
        .noneMatch(t -> t.score() < 50);
    System.out.println(noUnder50);
}
```

実行結果

```
false
```

ポイント

- !anyMatch(…) と同じ意味だが、「〜がいない」をそのまま書ける
- 条件を満たす要素が見つかった時点で止まる

演習 ・ 終端操作 ・ noneMatch

Bチームに50点未満はいないか

Bチームに50点未満の研修生が1人もいないことを判定し、結果を出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
true
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ noneMatch

Bチームに50点未満はいないか

```
{
    boolean ok = trainees.stream()
        .filter(t -> t.team().equals("B"))
        .noneMatch(t -> t.score() < 50);
    System.out.println(ok);
}
```

実行結果

```
true
```

解説

・「いないこと」を確かめたいので noneMatch

・Bチームは鈴木さん65点・伊藤さん77点なので true

終端操作 ・ count

count：要素の数を数える

要素の数を long で返します。

`long count()`

```
{
    long teamB = trainees.stream()
        .filter(t -> t.team().equals("B"))
        .count();
    System.out.println(teamB);
}
```

実行結果

2

ポイント

・filter と組み合わせて「条件に合う件数」を数えることが多い

・戻り値は int ではなく long

演習 ・ 終端操作 ・ count

70点以上の人数

70点以上の研修生の人数を求めて出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

4

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ count

70点以上の人数

```
{
    long passed = trainees.stream()
        .filter(t -> t.score() >= 70)
        .count();
    System.out.println(passed);
}
```

実行結果

4

解説

・filter で絞り込み、count で数える

・カウンタ変数を書き換えずに件数が得られる

終端操作 ・ findFirst

findFirst：先頭の要素を1つ取り出す（1/2）

先頭の要素を Optional に入れて返します。要素が無ければ、空の Optional を返します。

`Optional<T> findFirst()`

```
{
    Optional<Trainee> firstC = trainees.stream()
        .filter(t -> t.team().equals("C"))
        .findFirst();
    System.out.println(firstC);
}
```

実行結果

```
Optional[Trainee[name=田中, team=C, score=48]]
```

findFirst：先頭の要素を1つ取り出す（2/2）

```
{
    Optional<Trainee> firstD = trainees.stream()
        .filter(t -> t.team().equals("D"))
        .findFirst();
    System.out.println(firstD);
}
```

実行結果

```
Optional.empty
```

ポイント

・見つからない可能性を、戻り値の型 Optional で表している

・Optional の扱い方は、後半の章で詳しく学ぶ

演習 ・ 終端操作 ・ findFirst

60点未満で最初に見つかる研修生

60点未満の研修生のうち、リストの先頭に近い1人の名前を取り出して出力してください。

ヒント：名前に変換してから findFirst すると Optional<String> になる

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
Optional[田中]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ findFirst

60点未満で最初に見つかる研修生

```
{
    Optional<String> name = trainees.stream()
        .filter(t -> t.score() < 60)
        .map(Trainee::name)
        .findFirst();
    System.out.println(name);
}
```

実行結果

```
Optional[田中]
```

解説

・filter → map → findFirst の順につなぐ

・結果は Optional[田中] と出力される

終端操作 ・ min / max

min / max：最小・最大の要素を取り出す（1/2）

並べ方（Comparator）で比べて、最小・最大の要素を Optional に入れて返します。

```
Optional<T> max(Comparator<T> comparator)
```

```
{
    Optional<Trainee> top = trainees.stream()
        .max(Comparator.comparing(Trainee::score));
    System.out.println(top);
}
```

実行結果

```
Optional[Trainee[name=高橋, team=A, score=91]]
```

終端操作 ・ min / max

min / max：最小・最大の要素を取り出す（2/2）

```
{
    Optional<Trainee> bottom = trainees.stream()
        .min(Comparator.comparing(Trainee::score));
    System.out.println(bottom);
}
```

実行結果

```
Optional[Trainee[name=田中, team=C, score=48]]
```

ポイント

- sorted して findFirst するより、意図が名前に表れる
- 要素が0件のこともあるので Optional を返す

演習 ・ 終端操作 ・ min / max

Bチームの最高点の研修生

Bチームの研修生のうち、最も点数が高い研修生を取り出して出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
Optional[Trainee[name=伊藤, team=B, score=77]]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ min / max

Bチームの最高点の研修生

```
{
    Optional<Trainee> topB = trainees.stream()
        .filter(t -> t.team().equals("B"))
        .max(Comparator.comparing(Trainee::score));
    System.out.println(topB);
}
```

実行結果

```
Optional[Trainee[name=伊藤, team=B, score=77]]
```

解説

・filter で Bチームに絞ってから max

・比べる基準は Comparator.comparing(Trainee::score)

終端操作 ・ reduce

reduce：要素を1つの値に畳み込む（1/2）

初期値から始めて、要素を1つずつ合成していき、最後に1つの値にまとめます。

```
T reduce(T identity, BinaryOperator<T> accumulator)
```

```
{
    int total = trainees.stream()
        .map(Trainee::score)
        .reduce(0, (sum, score) -> sum + score);
    System.out.println(total);
}
```

実行結果

433

ポイント

・初期値は「要素が0件のときの答え」

・合計だけなら mapToInt(…).sum() のほうが意図が明確

終端操作 ・ reduce

reduce：要素を1つの値に畳み込む（2/2）

前のスライドの (sum, score) -> sum + score が、1件ごとにどう呼ばれるかを追います。

呼び出し	sum（前までの結果）	score（次の要素）	戻り値
1回目	0（初期値）	82	82
2回目	82	65	147
3回目	147	91	238
4回目	238	48	286
5回目	286	77	363
6回目	363	70	433

演習 ・ 終端操作 ・ reduce

reduce で最高点を求める

reduce を使って、研修生の点数の最高点を求めて出力してください。

ヒント：2つの数の大きいほうは Math.max(a, b) で求められる

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

91

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ reduce

reduce で最高点を求める

```
{
    int best = trainees.stream()
        .map(Trainee::score)
        .reduce(0, (a, b) -> Math.max(a, b));
    System.out.println(best);
}
```

実行結果

91

解説

・前までの最大値と次の点数を比べ、大きいほうを残していく

・実際の開発では mapToInt(…).max() でもよい

終端操作 ・ toList

toList：List にまとめる

要素を List にまとめて返します（Java 16以降）。返される List は変更できません。

```
List<T> toList()
```

```
{
    List<String> names = trainees.stream()
        .map(Trainee::name)
        .toList();
    System.out.println(names);
    names.add("山田"); // 要素を追加してみる
}
```

実行結果

```
[佐藤, 鈴木, 高橋, 田中, 伊藤, 渡辺]
|  Exception java.lang.UnsupportedOperationException
|        …（スタックトレースは省略）
```

終端操作 ・ toList

toList：List にまとめる（ポイント）

ポイント

・結果を後から書き換えられないので、安心して受け渡せる

・変更したいときは new ArrayList<>(list) で詰め直す

・古い書き方 collect(Collectors.toList()) は、変更できるかどうかが保証されていない

演習 ・ 終端操作 ・ toList

Aチームの名前のリスト

Aチームの研修生の名前をリストにまとめて出力してください。

続けて、そのリストに "山田" を add して、何が起きるかを確かめてください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[佐藤, 高橋]
|  Exception java.lang.UnsupportedOperationException
|        …（スタックトレースは省略）
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ toList

Aチームの名前のリスト

```
{
    List<String> teamA = trainees.stream()
        .filter(t -> t.team().equals("A"))
        .map(Trainee::name)
        .toList();
    System.out.println(teamA);
    teamA.add("山田");
}
```

実行結果

```
[佐藤, 高橋]
|  Exception java.lang.UnsupportedOperationException
|        …（スタックトレースは省略）
```

解説

・toList の結果は変更できないので、add すると UnsupportedOperationException になる

終端操作 ・ collect

collect：Collectors を使って、好きな形にまとめる（1/2）

「まとめ方」の部品（Collector）を受け取り、その方法で結果を作ります。部品は Collectors クラスにそろっています。

```
R collect(Collector<T, A, R> collector)
```

```
{
    Set<String> teams = trainees.stream()
        .map(Trainee::team)
        .collect(Collectors.toSet());
    System.out.println(teams);
}
```

実行結果

```
[A, B, C]
```

ポイント

- toSet() は重複を許さない Set にまとめる
- Set には順番の保証がない

終端操作 ・ collect

collect：Collectors を使って、好きな形にまとめる（2/2）

Collectors には、よく使うまとめ方がそろっています。この後、1つずつ見ていきます。

Collectors のメソッド	まとめ方
toList() / toSet()	List / Set にする
joining(…)	文字列を連結する
groupingBy(…)	キーごとに分類する
partitioningBy(…)	true / false で2つに分ける
toMap(…)	キーと値の Map にする

演習 ・ 終端操作 ・ collect

75点以上の研修生がいるチーム

75点以上の研修生がいるチームの名前を、重複の無い Set にまとめて出力してください。

ヒント：Collectors.toSet() を使う

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[A, B]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect

75点以上の研修生がいるチーム

```
{
    Set<String> teams = trainees.stream()
        .filter(t -> t.score() >= 75)
        .map(Trainee::team)
        .collect(Collectors.toSet());
    System.out.println(teams);
}
```

実行結果

```
[A, B]
```

解説

・A（佐藤・高橋）と B（伊藤）が残る

・distinct().toList() でも似た結果になるが、Set は「重複が無い」ことを型で表せる

終端操作 ・ collect(joining)

joining：文字列を連結する（1/2）

文字列の要素を区切り文字でつなぎ、1つの String にします。前後に付ける文字も指定できます。

`Collectors.joining(区切り, 先頭, 末尾)`

```
{
    String csv = trainees.stream()
        .map(Trainee::name)
        .collect(Collectors.joining(","));
    System.out.println(csv);
}
```

実行結果

佐藤,鈴木,高橋,田中,伊藤,渡辺

終端操作 ・ collect(joining)

joining：文字列を連結する（2/2）

```
{
    String message = trainees.stream()
        .filter(t -> t.score() >= 70)
        .map(Trainee::name)
        .collect(Collectors.joining(
            "さん、", "合格者：", "さん"));
    System.out.println(message);
}
```

実行結果

合格者：佐藤さん、高橋さん、伊藤さん、渡辺さん

ポイント

- 要素が String である必要がある（map で変換してから使う）
- 最後の要素の後ろに区切り文字が付かないので、自分で取り除かなくてよい

演習 ・ 終端操作 ・ collect(joining)

点数の高い順に名前をつなぐ

研修生を点数の高い順に並べ、名前を " > " でつないだ1つの文字列にして出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

高橋 > 佐藤 > 伊藤 > 渡辺 > 鈴木 > 田中

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect(joining)

点数の高い順に名前をつなぐ

```
{
    String ranking = trainees.stream()
        .sorted(Comparator.comparing(Trainee::score).reversed())
        .map(Trainee::name)
        .collect(Collectors.joining(" > "));
    System.out.println(ranking);
}
```

実行結果

高橋 > 佐藤 > 伊藤 > 渡辺 > 鈴木 > 田中

解説

・sorted → map → collect(joining) の順につなぐ

終端操作 ・ collect(groupingBy)

groupingBy：キーごとに分類する

要素からキーを取り出し、同じキーの要素を List にまとめた Map を返します。

`Collectors.groupingBy(キーを取り出す処理)`

```
{
    Map<String, List<Trainee>> byTeam = trainees.stream()
        .collect(Collectors.groupingBy(Trainee::team));

    System.out.println(byTeam.keySet());
    System.out.println(byTeam.get("A"));
}
```

実行結果

```
[A, B, C]
[Trainee[name=佐藤, team=A, score=82], Trainee[name=高橋, team=A, score=91]]
```

終端操作 ・ collect(groupingBy)

groupingBy：キーごとに分類する（ポイント）

ポイント

・結果は Map<キー, List<要素>>

・Map の順番は保証されない（必要なら TreeMap などを指定する）

演習 ・ 終端操作 ・ collect(groupingBy)

70点台の研修生を取り出す

研修生を点数の10点単位（82点なら80）をキーにして分類してください。

分類した Map から、キー 70 の研修生のリストを取り出して出力してください。

ヒント：キーは t -> t.score() / 10 * 10 のように計算して作ってもよい

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
[Trainee[name=伊藤, team=B, score=77], Trainee[name=渡辺, team=C, score=70]]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect(groupingBy)

70点台の研修生を取り出す

```
{
    Map<Integer, List<Trainee>> byRange = trainees.stream()
        .collect(Collectors.groupingBy(t -> t.score() / 10 * 10));

    System.out.println(byRange.get(70));
}
```

実行結果

```
[Trainee[name=伊藤, team=B, score=77], Trainee[name=渡辺, team=C, score=70]]
```

解説

・キーは Integer になり、結果は Map<Integer, List<Trainee>>

・get(70) で70点台の研修生だけを取り出せる

終端操作 ・ collect(groupingBy)

groupingBy：グループごとのまとめ方を変える（1/2）

第2引数に Collector を渡すと、各グループの中身を List 以外の形にまとめられます。

`Collectors.groupingBy(キー, まとめ方)`

```
{
    Map<String, Long> countByTeam = trainees.stream()
        .collect(Collectors.groupingBy(
            Trainee::team,
            Collectors.counting()));
    System.out.println(countByTeam);
}
```

実行結果

```
{A=2, B=2, C=2}
```

終端操作 ・ collect(groupingBy)

groupingBy：グループごとのまとめ方を変える（2/2）

```
{
    Map<String, List<String>> namesByTeam = trainees.stream()
        .collect(Collectors.groupingBy(
            Trainee::team,
            Collectors.mapping(
                Trainee::name, Collectors.toList())));
    System.out.println(namesByTeam);
}
```

実行結果

```
{A=[佐藤, 高橋], B=[鈴木, 伊藤], C=[田中, 渡辺]}
```

ポイント

・counting()：件数、averagingInt(…)：平均、summingInt(…)：合計

・mapping(変換, まとめ方)：変換してからまとめる

演習 ・ 終端操作 ・ collect(groupingBy)

チームごとの平均点

チームごとの平均点を求め、Map にして出力してください。

ヒント：平均は Collectors.averagingInt(Trainee::score)

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
{A=86.5, B=71.0, C=59.0}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect(groupingBy)

チームごとの平均点

```
{
    Map<String, Double> avgByTeam = trainees.stream()
        .collect(Collectors.groupingBy(
            Trainee::team,
            Collectors.averagingInt(Trainee::score)));
    System.out.println(avgByTeam);
}
```

実行結果

```
{A=86.5, B=71.0, C=59.0}
```

解説

・平均値は Double になるので、Map<String, Double>

終端操作 ・ collect(partitioningBy)

partitioningBy：条件で2つに分ける

条件の true / false で要素を2つに分け、Map<Boolean, List<T>> にします。第2引数でまとめ方も指定できます。

`Collectors.partitioningBy(条件, まとめ方)`

```
{
    Map<Boolean, List<String>> result = trainees.stream()
        .collect(Collectors.partitioningBy(
            t -> t.score() >= 70,
            Collectors.mapping(
                Trainee::name, Collectors.toList())));

    System.out.println(result.get(true));
    System.out.println(result.get(false));
}
```

実行結果

```
[佐藤, 高橋, 伊藤, 渡辺]
[鈴木, 田中]
```

終端操作 ・ collect(partitioningBy)

partitioningBy：条件で2つに分ける（ポイント）

ポイント

・キーは必ず true と false の2つ。該当が0件でも空のリストが入る

・「合格・不合格」のように2つに分けるなら groupingBy よりこちら

演習 ・ 終端操作 ・ collect(partitioningBy)

平均点以上・未満の人数

全員の平均点を求めてください。

平均点以上かどうかで研修生を2つに分け、それぞれの人数を Map にして出力してください。

ヒント：人数は Collectors.counting()

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
{false=3, true=3}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect(partitioningBy)

平均点以上・未満の人数

```
{
    double avg = trainees.stream()
        .mapToInt(Trainee::score)
        .average()
        .orElse(0.0);

    Map<Boolean, Long> result = trainees.stream()
        .collect(Collectors.partitioningBy(
            t -> t.score() >= avg,
            Collectors.counting()));
    System.out.println(result);
}
```

実行結果

```
{false=3, true=3}
```

解答例 ・ 終端操作 ・ collect(partitioningBy)

平均点以上・未満の人数（解説）

解説

・平均点は約72.2点。82・91・77点の3人が true

・ラムダ式の中で使う avg は、書き換えない変数なので参照できる

終端操作 ・ collect(toMap)

toMap：キーと値の Map にする（1/2）

各要素からキーと値を取り出して Map を作ります。キーが重複したときの合成方法も指定できます。

`Collectors.toMap(キー, 値, 重複時の合成)`

```
{
    Map<String, Integer> scoreByName = trainees.stream()
        .collect(Collectors.toMap(
            Trainee::name, Trainee::score));
    System.out.println(scoreByName.get("高橋"));
}
```

実行結果

```
91
```

終端操作 ・ collect(toMap)

toMap：キーと値の Map にする（2/2）

```
{
    Map<String, String> namesByTeam = trainees.stream()
        .collect(Collectors.toMap(
            Trainee::team,
            Trainee::name,
            (a, b) -> a + "," + b)); // キー重複時の合成
    System.out.println(namesByTeam);
}
```

実行結果

```
{A=佐藤,高橋, B=鈴木,伊藤, C=田中,渡辺}
```

ポイント

・合成方法を指定せずにキーが重複すると例外になる：IllegalStateException: Duplicate key A

・キーが一意（名前など）なら、第3引数は不要

演習 ・ 終端操作 ・ collect(toMap)

チームごとの最高点

チームをキー、そのチームの最高点を値とする Map を作り、出力してください。

ヒント：同じチームが2回出てきたら、大きいほうの点数を残す

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
{A=91, B=77, C=70}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ collect(toMap)

チームごとの最高点

```
{
    Map<String, Integer> bestByTeam = trainees.stream()
        .collect(Collectors.toMap(
            Trainee::team,
            Trainee::score,
            (a, b) -> Math.max(a, b)));
    System.out.println(bestByTeam);
}
```

実行結果

```
{A=91, B=77, C=70}
```

解説

・重複時の合成で Math.max を使い、大きいほうを残す

終端操作 ・ forEach

forEach：1件ずつ処理する

各要素に処理を行います。戻り値はありません（void）。

```
void forEach(Consumer<T> action)
```

```
{
    trainees.stream()
        .filter(t -> t.score() < 60)
        .forEach(t -> System.out.println(t.name() + "さんは再テスト"));
}
```

実行結果

田中さんは再テスト

ポイント

・画面に出す・保存するなど、結果を外に出す最後の場面で使う

・forEach の中で外のリストに add しない（toList や collect で受け取る）

演習 ・ 終端操作 ・ forEach

優秀賞を表示する

80点以上の研修生について、「佐藤さん 優秀賞」の形式で1人ずつ出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

佐藤さん　優秀賞
高橋さん　優秀賞

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 終端操作 ・ forEach

優秀賞を表示する

```
{
    trainees.stream()
        .filter(t -> t.score() >= 80)
        .forEach(t -> System.out.println(t.name() + "さん 優秀賞"));
}
```

実行結果

佐藤さん 優秀賞
高橋さん 優秀賞

解説

・filter で絞ってから forEach で出力する

06

Stream の組み合わせ演習

複数のメソッドをつないで、やりたいことを書く

演習 ・ 組み合わせ演習 1

合格者を点数の高い順に並べる

70点以上の研修生を、点数の高い順に並べてください。

名前を「、」でつないだ1つの文字列にして出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

高橋、佐藤、伊藤、渡辺

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 組み合わせ演習 1

合格者を点数の高い順に並べる

```
{
    String result = trainees.stream()
        .filter(t -> t.score() >= 70)
        .sorted(Comparator.comparing(Trainee::score).reversed())
        .map(Trainee::name)
        .collect(Collectors.joining("、"));
    System.out.println(result);
}
```

実行結果

高橋、佐藤、伊藤、渡辺

解説

- filter → sorted → map → joining
- 絞り込みを先にすると、並べ替える件数が減る

演習 ・ 組み合わせ演習 2

チームごとの合格者

70点以上の研修生だけを対象に、チームごとに名前のリストを作り、Map にして出力してください。

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
{A=[佐藤, 高橋], B=[伊藤], C=[渡辺]}
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 組み合わせ演習 2

チームごとの合格者

```
{
    Map<String, List<String>> passedByTeam = trainees.stream()
        .filter(t -> t.score() >= 70)
        .collect(Collectors.groupingBy(
            Trainee::team,
            Collectors.mapping(Trainee::name, Collectors.toList())));
    System.out.println(passedByTeam);
}
```

実行結果

```
{A=[佐藤, 高橋], B=[伊藤], C=[渡辺]}
```

解説

- filter で絞ってから groupingBy
- mapping で Trainee を名前に変換してからリストにする
- 合格者がいないチームはキー自体が作られない

演習 ・ 組み合わせ演習 3

研修完了の判定

次の2つを両方満たすなら「研修完了」、そうでなければ「補講あり」と出力してください。

1. 全員が40点以上である　2. 全員の平均点が70点以上である

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

研修完了

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 組み合わせ演習 3

研修完了の判定

```
{
    boolean allOver40 = trainees.stream()
        .allMatch(t -> t.score() >= 40);
    double avg = trainees.stream()
        .mapToInt(Trainee::score)
        .average()
        .orElse(0.0);

    String status = (allOver40 && avg >= 70) ? "研修完了" : "補講あり";
    System.out.println(status);
}
```

実行結果

研修完了

解説

- 判定は allMatch、平均は mapToInt → average
- 2つの結果を変数で受け取ってから組み合わせると読みやすい

演習 ・ 組み合わせ演習 4

ログを集計する

次のログ一覧を使います。各ログは「レベル 半角空白 メッセージ」の形式です。

List.of("ERROR 接続失敗", "INFO 起動", "WARN 応答遅延", "ERROR タイムアウト", "INFO 停止")

1. レベルごとの件数を Map にして出力してください。

2. ERROR のメッセージ部分だけをリストにして出力してください。

ヒント：log.split(" ")[0] でレベル、[1] でメッセージを取り出せる

期待する結果

```
{ERROR=2, INFO=2, WARN=1}
[接続失敗, タイムアウト]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 組み合わせ演習 4

ログを集計する

```
{
    List<String> logs = List.of("ERROR 接続失敗", "INFO 起動",
        "WARN 応答遅延", "ERROR タイムアウト", "INFO 停止");

    Map<String, Long> countByLevel = logs.stream()
        .collect(Collectors.groupingBy(
            log -> log.split(" ")[0],
            Collectors.counting()));
    System.out.println(countByLevel);

    List<String> errors = logs.stream()
        .filter(log -> log.startsWith("ERROR"))
        .map(log -> log.split(" ")[1])
        .toList();
    System.out.println(errors);
}
```

実行結果

```
{ERROR=2, INFO=2, WARN=1}
[接続失敗, タイムアウト]
```

解答例 ・ 組み合わせ演習 4

ログを集計する（解説）

解説

- groupingBy のキーは、要素から計算して作れる
- startsWith で前方一致を判定できる

演習 ・ 組み合わせ演習 5

2番目に点数が高い研修生

2番目に点数が高い研修生の名前を求めて出力してください。

ヒント：高い順に並べて、先頭の1人を飛ばす

trainees　佐藤 A 82 ／ 鈴木 B 65 ／ 高橋 A 91 ／ 田中 C 48 ／ 伊藤 B 77 ／ 渡辺 C 70

期待する結果

```
Optional[佐藤]
```

JShell に貼り付けて実行し、結果を確かめます。複数行のコードは、全体を { } で囲んで貼り付けます。

解答例 ・ 組み合わせ演習 5

2番目に点数が高い研修生

```
{
    Optional<String> second = trainees.stream()
        .sorted(Comparator.comparing(Trainee::score).reversed())
        .skip(1)
        .map(Trainee::name)
        .findFirst();
    System.out.println(second);
}
```

実行結果

```
Optional[佐藤]
```

解説

- sorted → skip(1) → findFirst
- 研修生が1人しかいない場合もあるので、結果は Optional になる

まとめ

Stream API：やりたいことを、メソッド名で書く

意図が読める

filter・map・groupingBy などのメソッド名が、そのまま「何をしたいか」を表す。手順を頭でなぞらなくてよい。

書き換えない

元のリストも途中の変数も書き換えず、結果を新しい値として受け取る。値の移り変わりを追わなくてよい。

必要な分だけ動く

中間操作は終端操作を呼ぶまで実行されず、要素ごとに処理され、答えが決まれば止まる。

迷ったら戻り値の型を見る。Stream を返せば中間操作、それ以外を返せば終端操作です。

まとめ－やりたいこと別 早見表

中間操作（1/3）

やりたいこと	書き方	戻り値
条件に合うものだけ残す	filter(t -> 条件)	Stream<T>
別の値に変換する	map(t -> 値)	Stream<R>
並べ替える	sorted(Comparator.comparing(…))	Stream<T>
逆順に並べ替える	sorted(Comparator.comparing(…).reversed())	Stream<T>
重複を除く	distinct()	Stream<T>
先頭から n 件だけ残す	limit(n)	Stream<T>
先頭の n 件を飛ばす	skip(n)	Stream<T>
ネストしたリストを平らにする	flatMap(x -> x.stream())	Stream<R>
数値として集計できるようにする	mapToInt(t -> 数値)	IntStream

まとめ－やりたいこと別 早見表

終端操作：判定・取り出し・集計（2/3）

やりたいこと	書き方	戻り値
1つでも当てはまるか	anyMatch(t -> 条件)	boolean
すべて当てはまるか	allMatch(t -> 条件)	boolean
どれも当てはまらないか	noneMatch(t -> 条件)	boolean
件数を数える	count()	long
先頭の1件を取り出す	findFirst()	Optional<T>
最大・最小を取り出す	max(…) / min(…)	Optional<T>
合計する	mapToInt(…).sum()	int
平均する	mapToInt(…).average()	OptionalDouble
1つの値に畳み込む	reduce(初期値, (a, b) -> …)	T
List にまとめる	toList()	List<T>
1件ずつ処理する	forEach(t -> 処理)	void

まとめ－やりたいこと別 早見表

終端操作：collect でまとめる（3/3）

やりたいこと	書き方	戻り値
Set にまとめる	collect(Collectors.toSet())	Set<T>
文字列をつなぐ	joining(区切り)	String
前後を付けて文字列をつなぐ	joining(区切り, 先頭, 末尾)	String
キーごとに分類する	groupingBy(キー)	Map<K, List<T>>
分類して件数を数える	groupingBy(キー, counting())	Map<K, Long>
分類して変換してまとめる	groupingBy(キー, mapping(変換, toList()))	Map<K, List<R>>
分類して平均する	groupingBy(キー, averagingInt(…))	Map<K, Double>
条件で2つに分ける	partitioningBy(条件)	Map<Boolean, List<T>>
キーと値の Map にする	toMap(キー, 値)	Map<K, V>
キーの重複を合成する	toMap(キー, 値, (a, b) -> …)	Map<K, V>

※ joining・groupingBy などは Collectors のメソッドで、collect(Collectors.joining(…)) のように collect に渡して使います。